ADMINISTRACIÓN DE BASES DE DATOS

TEMA 5 DEL PROGRAMA FUNDAMENTOS DE NoSQL Y MONGODB OPERACIONES CRUD

Apuntes de estudio y prácticas de laboratorio

Ingeniería en Sistemas Computacionales · Nivel: introductorio con conocimientos previos de MySQL y Taller de Base de Datos

Propósito: comprender el modelo documental y ejecutar operaciones de administración básica y CRUD desde MongoDB Shell (mongosh).

Indicaciones: desarrollar los ejemplos en una base de datos de práctica local. Los comandos de borrado se ejecutan exclusivamente sobre datos de laboratorio.

Ruta de aprendizaje e índice

1. Modelo NoSQL: temas 1–8

2. Herramientas y estructuras: temas 9–16

3. Inserción y consultas: temas 17–18

4. Actualizaciones: temas 19–27

5. Eliminaciones: temas 28–30

6. Laboratorio integrador y autoevaluación

1. Modelo NoSQL · temas 1–8

1. ¿Qué es NoSQL?

Familia de tecnologías de bases de datos que no exige el modelo relacional tradicional. NoSQL no significa que SQL sea inútil: son alternativas para distintos requisitos.

Ejemplo: un catálogo en MongoDB puede conservar especificaciones distintas para cada producto.

Ejercicio: Explica dos razones para elegir documentos frente a filas y dos circunstancias donde MySQL podría convenir.

2. Tipos de NoSQL

Documentales: documentos BSON (MongoDB). Clave-valor: acceso por clave (Redis). Familia de columnas/columnas anchas: datos organizados por particiones y familias (Cassandra). Grafos: nodos y relaciones (Neo4j). No confundir las bases de columnas anchas con los formatos analíticos columnares.

Ejemplo: red social con amistades: grafo; perfil flexible: documental.

Ejercicio: Asocia cuatro escenarios con tipos de NoSQL y justifica tu elección.

3. ¿Qué es MongoDB?

Sistema de base de datos documental que almacena documentos BSON en colecciones. Consultamos mediante filtros y métodos de la API desde mongosh.

Ejemplo: una colección productos contiene documentos con nombre, precio y características.

Ejercicio: Escribe una definición propia de MongoDB y explica qué almacena una colección.

4. Características, ventajas y limitaciones

Permite estructuras flexibles, arreglos, documentos anidados e índices. Simplifica la representación de ciertos datos semiestructurados. Requiere diseño cuidadoso: duplicaciones, documentos enormes, validaciones y consultas entre colecciones pueden complicarse.

Ejemplo: guardar direcciones embebidas cuando se consultan con un cliente.

Ejercicio: Describe dos ventajas y dos riesgos concretos de utilizar documentos.

5. Diferencias entre MySQL y MongoDB

MySQL trabaja principalmente con tablas, filas, esquema y SQL. MongoDB trabaja con colecciones, documentos y consultas por objetos. Las relaciones en MongoDB pueden modelarse con referencias o incrustaciones; las consultas entre colecciones existen mediante agregación ($lookup), pero no son equivalentes directas a JOIN en diseño y costo. Ambos sistemas admiten índices y transacciones bajo condiciones específicas.

MySQL: SELECT * FROM productos; MongoDB: db.productos.find().

Ejercicio: Convierte un SELECT con WHERE a una consulta find() y explica el filtro.

6. Bases de datos, colecciones y documentos

Base de datos: contenedor de colecciones. Colección: conjunto de documentos. Documento: estructura de pares campo-valor, potencialmente anidada. Un mismo conjunto puede alojar campos diferentes entre documentos.

Ejemplo: tienda_mongo → productos → {nombre:"Mouse", precio:180}.

Ejercicio: Dibuja la jerarquía de una base, tres colecciones y dos documentos.

7. JSON y BSON

JSON es una notación textual. BSON es una representación binaria usada por MongoDB que admite tipos como ObjectId, Date y Decimal128. Lo que tecleamos en mongosh se parece a JSON, pero usa sintaxis de JavaScript y extensiones BSON.

Ejemplo: fecha: ISODate("2026-10-20T10:00:00Z").

Ejercicio: Explica por qué ObjectId y Date no son tipos JSON estándares.

8. _id y ObjectId

Cada documento tiene un campo _id único dentro de su colección. Si se omite, normalmente se genera automáticamente un ObjectId. _id no tiene que ser ObjectId, pero debe ser único; el índice único se crea sobre _id. ObjectId contiene 12 bytes y no debe tratarse como prueba exacta de fecha de creación.

Ejemplo: db.productos.findOne({}, {_id:1}).

Ejercicio: Inserta un documento sin _id y observa el valor generado.

2. Herramientas y estructura · temas 9–16

9. Server, mongosh y Compass

MongoDB Server (mongod) es el servidor que almacena los datos. mongosh es el intérprete de comandos; Compass es una interfaz gráfica. Compass no sustituye al servidor.

Ejercicio: Identifica qué herramienta almacena datos, cuál ejecuta comandos y cuál permite explorarlos visualmente.

10. Conexión al servidor

```
mongosh "mongodb://127.0.0.1:27017"

// Con autenticación: configurar credenciales en un entorno seguro
```

Ejecuta mongosh desde la terminal del sistema operativo, no dentro de una sesión mongosh existente. Confirma que el servidor esté iniciado. Evita publicar contraseñas en capturas o scripts.

Ejercicio: Conecta al servidor local y registra el nombre de la base de datos activa usando db.getName().

11. Listado de bases de datos

```
show dbs

db.adminCommand({listDatabases:1})
```

show dbs lista bases visibles según permisos; bases vacías pueden no aparecer.

Ejercicio: Lista las bases y comprueba cuáles son del sistema.

12. Selección y creación de bases

```
use tienda_mongo

db.getName()
```

use selecciona una base y no garantiza su creación inmediata; se materializa cuando se guardan datos o se crea una colección.

Ejercicio: Selecciona laboratorio5 y comprueba si aparece antes y después de insertar un documento.

13. Crear y eliminar colecciones

```
db.createCollection("productos")

show collections

db.productos.drop()

// PELIGRO: db.dropDatabase() elimina toda la base activa
```

También puede crearse una colección al insertar el primer documento. drop() elimina la colección y sus datos e índices; dropDatabase() elimina la base seleccionada.

Ejercicio: Crea una colección temporal, insértale un documento y elimínala; verifica cada paso.

14. Diferentes tipos de datos

```
db.productos.insertOne({

 codigo:"P01",nombre:"Mouse",precio:180.5,

 existencia:30,activo:true,fechaAlta:new Date(),

 descuento:null

})
```

Se muestran cadena, número, booleano, fecha y null. Para importes monetarios exactos se recomienda evaluar Decimal128 en lugar de flotantes binarios.

Ejercicio: Crea un producto con al menos seis campos de tipos distintos y revisa con typeof en mongosh.

15. Documentos anidados y arreglos

```
db.clientes.insertOne({

 clienteId:1,nombre:"Ana",

 direccion:{ciudad:"Pachuca",cp:"42000"},

 telefonos:["7711111111","7712222222"]

})
```

El campo direccion es un documento embebido y telefonos es un arreglo. Los accesos a campos anidados usan notación de puntos.

Ejercicio: Crea otro cliente con dirección y dos teléfonos; consúltalo por ciudad.

16. Consultas básicas de estructura

```
show collections

db.productos.findOne()

db.productos.countDocuments({})

db.productos.getIndexes()

db.productos.findOne({}, {_id:0,nombre:1})
```

Inspecciona colecciones, documentos, cantidad e índices. MongoDB no exige que todos los documentos compartan los mismos campos; getIndexes() muestra índices, no una descripción SQL de columnas.

Ejercicio: Identifica campos presentes en dos documentos distintos y señala diferencias.

3. Operaciones CREATE y READ · temas 17–18

Preparación del laboratorio: datos de ejemplo

```
use tienda_mongo

db.productos.drop()

db.clientes.drop()

db.ventas.drop()

db.productos.insertMany([

 {codigo:"P01",nombre:"Mouse",precio:180,existencia:30,categoria:"Accesorios",etiquetas:["usb","oferta"]},

 {codigo:"P02",nombre:"Teclado",precio:350,existencia:15,categoria:"Accesorios",etiquetas:["usb"]},

 {codigo:"P03",nombre:"Monitor",precio:2800,existencia:8,categoria:"Pantallas",etiquetas:["hdmi"]},

 {codigo:"P04",nombre:"Memoria USB",precio:150,existencia:0,categoria:"Almacenamiento",etiquetas:["usb"]}

])
```

¡Importante! El script borra las colecciones nombradas antes de sembrar datos. Usarlo solamente en la base de prácticas tienda_mongo.

17. insertOne() e insertMany()

```
db.clientes.insertOne({clienteId:1,nombre:"Ana",ciudad:"Pachuca"})

db.clientes.insertMany([

 {clienteId:2,nombre:"Luis",ciudad:"Tulancingo"},

 {clienteId:3,nombre:"Sara",ciudad:"Pachuca"}

])
```

Sintaxis: db.coleccion.insertOne(documento) y db.coleccion.insertMany([documentos]). Devuelven identificadores insertados; si falta _id se genera automáticamente. No garantizan unicidad en clienteId sin índice único adicional.

Ejercicio: Inserta dos productos y explica la diferencia entre insertar uno y varios documentos.

18. find(), findOne(), filtros y proyecciones

```
db.productos.find({})

db.productos.findOne({codigo:"P01"})

db.productos.find({precio:{$gt:200}}, {_id:0,nombre:1,precio:1})

db.productos.find({existencia:{$lte:10}})

db.productos.find({categoria:{$in:["Pantallas","Accesorios"]}})

db.productos.find({$or:[{precio:{$lt:200}},{existencia:0}]})

db.productos.find({etiquetas:"usb"})

db.productos.find().sort({precio:-1}).limit(2)

db.productos.countDocuments({categoria:"Accesorios"})
```

Sintaxis: db.coleccion.find(filtro, proyeccion). Operadores: $eq (=), $ne (≠), $gt (>), $gte (≥), $lt (<), $lte (≤), $in, $nin; lógicos $and, $or, $not. Orden: 1 ascendente, -1 descendente. Proyección: 1 incluir, 0 excluir; _id se excluye explícitamente.

Ejercicio: Obtén a) productos de precio mayor o igual a 180; b) productos agotados; c) los dos productos de mayor precio sin mostrar _id; d) productos con etiqueta usb.

4. Operaciones UPDATE · temas 19–27

19. updateOne()

```
db.productos.updateOne({codigo:"P01"},{$set:{precio:200}})
```

Modifica un documento que cumpla el filtro. Devuelve matchedCount y modifiedCount; si no hay coincidencia no modifica registros.

Ejercicio: Cambia la existencia de P02 a 18 y comprueba resultado.

20. updateMany()

```
db.productos.updateMany({categoria:"Accesorios"},{$inc:{existencia:2}})
```

Modifica todos los documentos que cumplan el filtro. Un filtro vacío {} puede afectar a toda la colección.

Ejercicio: Incrementa en 3 la existencia de productos con existencia menor a 20.

21. $set

```
db.productos.updateOne({codigo:"P03"},{$set:{marca:"Ejemplo",precio:2750}})
```

$set actualiza campos existentes o agrega campos nuevos sin eliminar los demás.

Ejercicio: Agrega un campo garantiaMeses al producto P01.

22. $inc

```
db.productos.updateOne({codigo:"P01"},{$inc:{existencia:-2}})
```

$inc incrementa un campo numérico usando cantidades positivas o negativas. No se aplica a campos con valor null.

Ejercicio: Simula venta de tres unidades de P02, verificando antes que hay suficientes existencias.

23. $unset

```
db.productos.updateOne({codigo:"P03"},{$unset:{marca:""}})
```

$unset elimina un campo de un documento; no elimina el documento ni la colección.

Ejercicio: Agrega campo temporal a P01 y después elimínalo.

24. $push

```
db.productos.updateOne({codigo:"P01"},{$push:{etiquetas:"inalambrico"}})
```

$push añade un elemento a un arreglo; puede repetir valores. Para evitar duplicados es útil conocer $addToSet (ampliación).

Ejercicio: Agrega una etiqueta nueva a P02 y observa el arreglo resultante.

25. $pull

```
db.productos.updateOne({codigo:"P01"},{$pull:{etiquetas:"oferta"}})
```

$pull elimina del arreglo todos los elementos que satisfacen el criterio.

Ejercicio: Retira usb de las etiquetas de P02 y comprueba el efecto.

26. Modificar frente a reemplazar

```
db.productos.updateOne({codigo:"P04"},{$set:{precio:165}})
```

Con operadores de actualización se conservan los campos no mencionados; al reemplazar se define de nuevo el cuerpo completo, salvo _id.

Ejercicio: Explica por qué un reemplazo incompleto puede hacer perder información.

27. replaceOne()

```
db.productos.replaceOne({codigo:"P04"},{codigo:"P04",nombre:"Memoria
USB",precio:165,existencia:0,categoria:"Almacenamiento",etiquetas:["usb"]})
```

replaceOne(filtro, reemplazo) sustituye el documento coincidente, conservando su _id; no se permiten operadores como $set dentro del reemplazo.

Ejercicio: Reemplaza un documento de prueba con todos sus campos; compara antes y después.

5. Operaciones DELETE · temas 28–30

28. deleteOne()

```
db.productos.find({codigo:"P04"})

db.productos.deleteOne({codigo:"P04"})
```

Elimina como máximo un documento coincidente. Siempre verifica antes el filtro.

Ejercicio: Crea un documento temporal, consúltalo y elimínalo con deleteOne().

29. deleteMany()

```
db.productos.find({existencia:0})

db.productos.deleteMany({existencia:0})
```

Elimina todos los documentos coincidentes; deleteMany({}) borraría todos los documentos de la colección.

Ejercicio: Inserta dos productos de prueba con activo:false y elimina solo esos dos.

30. Eliminar documentos frente a eliminar colección

```
db.productos.deleteMany({}) // colección e índices permanecen

db.productos.drop()      // colección e índices desaparecen
```

deleteMany({}) vacía documentos, mientras drop() elimina la colección; no ejecutar estos comandos sobre datos productivos.

Ejercicio: En una colección temporal compara los resultados de show collections antes y después de deleteMany({}) y de drop().

6. Práctica integradora y autoevaluación

Proyecto: elaborar una pequeña tienda con colecciones clientes, productos y ventas. En ventas usa una referencia clienteId y un arreglo de conceptos: [{codigo:"P01",cantidad:2,precioUnitario:180}]. Explica que cada precioUnitario representa el precio en el momento de la venta.

1. Crear tienda_final y las tres colecciones.

2. Insertar 5 clientes, 8 productos y 4 ventas, con datos consistentes.

3. Escribir 8 consultas usando igualdad, rango, $in, $or, proyección, sort y limit.

4. Ejecutar 2 updateOne y 2 updateMany con $set y $inc.

5. Demostrar $unset, $push y $pull con una lista de etiquetas.

6. Comparar updateOne con replaceOne en una copia de prueba.

7. Ejecutar un deleteOne y un deleteMany con filtros verificados mediante find.

8. Mostrar la diferencia entre deleteMany({}) y drop() en colección temporal.

9. Documentar 5 equivalencias y 3 diferencias con MySQL.

10. Entregar script ejecutable, evidencias de salida y una reflexión breve sobre diseño documental.

Preguntas para revisar lo aprendido

• ¿Cuándo se materializa una base seleccionada con use?

• ¿Qué diferencia hay entre BSON y JSON?

• ¿Para qué sirve _id? ¿Debe ser siempre ObjectId?

• ¿Qué parámetros recibe find()?

• ¿Por qué conviene usar find() antes de updateMany() o deleteMany()?

• ¿En qué difieren $set y replaceOne()?

• ¿Qué efecto tienen $push y $pull sobre arreglos?

• ¿Por qué drop() no es equivalente a deleteMany({})?

Fuentes oficiales para consulta

MongoDB Docs: https://www.mongodb.com/docs/manual/ · CRUD: https://www.mongodb.com/docs/manual/crud/ · Documentos: https://www.mongodb.com/docs/manual/core/document/ · mongosh: https://www.mongodb.com/docs/mongodb-shell/